

Swift is a general purpose programming language for developing iOS applications. It is developed by Apple Inc.

What is Swift?

Swift is a general purpose, multi-paradigm, compiled programming language for developing iOS and OS X tvOS, watchOS applications. It is developed by Apple Inc. It is powerful and intuitive language which is easy to learn. Swift code is safe, precise and runs very fast.

Swift follows Objective-C runtime library which allows C, Objective-C, C++ and Swift code to run within one program. Swift is built with open source LLVM compiler and included in Xcode since version 6.

History of Swift Programming Language

Swift was introduced in 2014 at Apples's Worldwide Developers Conference.

The development of Swift was started in 2010 by Chris Lattner with other programmers i.e. Doug Gregor, John McCall, Ted Kremenek, and Joe Groff at Apple. The basic idea of Swift was taken from Objective-C, Rust, Haskell, Ruby, Python, C#, CLU, and many other programming languages.

Swift first version was 1.0 which undergone a major upgrade to Swift 2 at WWDC 2015. Its upgraded version 2.2 was made open-source software under the Apache License 2.0 on December 3, 2015, for Apple's platforms and Linux.

Its third version, Swift 3.0 went through a significant evolution where its syntax was changed. Swift overcame the popularity of Objective C in the first quarter of 2018.

Swift gone through several changes in Swift 4.0 version where you can update the code written with previous versions of Swift using the migration functionality built into Xcode.

Swift Version History

Version	Released Date
Swift 1.0	2014-09-09
Swift 1.1	2014-10-22
Swift 1.2	2015-04-08
Swift 2.0	2015-09-21
Swift 3.0	2016-09-13
Swift 4.0	2017-09-19
Swift 4.1	2018-03-29
Swift 4.2	2018-09-17

Swift Syntax

Comments in Swift

Comments are used in programs to make them clearly understandable. They are like helping texts in programs and ignored by compiler. In Swift 4, single line comments are written using `//` at the beginning of the comment.

Single line comment in Swift 4:

```
// This is a single line comment.
```

Multi-line comment in Swift 4:

Multi-line comments start with `/*` and end with the characters `*/` as shown below -

```
/* This is multiline comment */
```

Multi-line comments can be nested in Swift 4. i.e.

```
/* This is a multi-line comment.
```

```
/* This is the second line. */ */
```

Semicolons in Swift

In Swift 4, you don't need to type a semicolon (`;`) in your code as a closing statement. Though, it is optional, you can use it without any problem. If you

are using multiple statements in the same line, then you have to use a semicolon as a delimiter, otherwise the compiler will raise a syntax error.

For example,

```
/* First Swift 4 program */  
var myString = "Hello, World!"; print(myString)
```

Without using semicolon:

```
/* First Swift 4 program */  
var myString = "Hello, World!"  
print(myString)
```

Identifiers in Swift

In Swift 4, identifiers are used to identify a variable, function, or any other user-defined item. Swift 4 identifiers start with an alphabet A to Z or a to z or an underscore _ followed by zero or more letters, underscores, and digits (0 to 9).

In Swift 4, we can't use special characters such as @, \$, and % within identifiers. Swift 4 is a case sensitive programming language, so Literal and literal are two different identifiers.

These are some example of acceptable identifiers:

Ajeet sonoo ak_47

If you want to use a reserved word as an identifier you will have to put a backtick (`) before and after that reserved word. For example, class is not a valid identifier, but `class` is valid.

Reserved Keywords in Swift

In Swift 4, the reserved keywords can't be used as constants or variables or any other identifier names. If you want to use them as identifiers, you will use them within backticks (`).

Keywords used in declarations

Class	Func	Let	public
deinit	Enum	extension	import

Init	internal	operator	private
protocol	static	struct	subscript
typealias	var		

Keywords used in Statements

break	case	continue	default
do	else	fallthrough	for
if	in	return	switch
where	while		

Keywords used in expressions and types

as	dynamicType	false	is
nil	self	Self	super
true	_COLUMN_	_FILE_	_FUNCTION_
LINE			

Keywords used in particular contexts

associativity	convenience	dynamic	didSet
final	get	infix	inout
lazy	left	mutating	none
nonmutating	optional	override	postfix
precedence	prefix	Protocol	required
right	set	Type	unowned
weak	willSet		

Whitespaces in Swift

In Swift 4, whitespace is used to describe blanks, tabs, newline characters, and comments. It separates one part of the statement from another. It makes the computer to identify that here one element ends and another begins.

For example

```
var age
```

We have to put at least one whitespace character (usually a space) between var and age to make compiler to distinguish them.

On the other hand, in the following statement -

```
int courses = html + css //discount on the combined course
```

No whitespace character is necessary between courses and =, or between = and html, although you can include them for better readability.

You should make the equal space on both side of the operator.

For example

```
int courses = html + css //Correct statement
```

```
int courses= html+ css //Incorrect statement
```

The Swift 4 compiler ignores blank line which contains only whitespace.

Literals in Swift

A literal is used to represent the source code of a value of an integer, floating-point number, or string type.

For example

Integer literal

26

Floating-point literal

3.14159

String literal

"Hello, JavaTpoint!"

Print Statement in Swift

In Swift4, the 'print' keyword is used to print anything. There are three different properties of print keyword.

Items: Items that you want to print.

Separator: Used to separate items.

Terminator: It specifies the last value where the line ends.

For example

```
print("Items you want to print", separator: "Value " , terminator: "Value")
```

```
// E.g. of print statement.
```

```
print("Value one")
```

```
// prints "Value one \n" Adds, \n as terminator and " " as separator by default.
```

```
print("Value one","Value two", separator: " Next Value" , terminator: " End")
```

```
//prints "Value one Next Value Value two End"
```

The first print statement adds \n , newline Feed as terminator by default, where as in second print statement we've given " End " as terminator, hence it'll print "End " instead of \n.

We can use custom separator and terminators according to our requirement.

Data Types in Swift

Data Types	Example	Description
Character	"s","a"	a 16-bit Unicode character
String	"hello world!"	represents textual data
Int	3, -23	an integer number
Float	2.4, 3.14, -23.21	represents 32-bit floating-point number
Double	2.422342412414	represents 64-bit floating-point number
Bool	true and false	Any of two values: true or false

In Swift 4 programming language, data types are used while creating a variable or a constant. In every programming language, we have to use different types of variables to store data. Variables are reserved memory locations for storing values. This means when you create a variable, you reserve some space in memory.

A data type is a type of data (value) a variable can store in it. This data can be a text/string ("JavaTpoint") or a number (12.34) or just bits (0 & 1). We have to define the data type to ensure that only defined type of data is stored.

For example,

Suppose you have to store data of a student where you need to store his name and marks. The mark is a number (i.e. 70) and the name is a string (i.e. Alex) so, you need to create two variables or constants to store the data.

In Swift 4, you can do this by declaring variables and the data type as:

```
var Marks:Int = 70
```

```
var Name:String = "Alex"
```

Here, we declared Marks variable of type Int which stores value 70 and, Name variable of type String which stores value Alex.

You will get a compile time error if you use the following syntax:

```
var Score:Int = "Alex"
```

It will show a message stating cannot convert value of type 'String' to specify type 'Int'.

It shows that a number can only be stored in Int data type not in String data type.

Size of a Data Type

Size of the data type specifies the size of data that can be stored in a given variable or constant. The data type's size is measured in bit and it can store a value up to 2bits.

So, data type size 1 bit can store upto $2^1 = 2$ values: either 0 or 1.

0 -> a or 0

1 -> b or 1

Data type size 2 bit can store up to $2^2 = 4$ values. Which are: (00,01,10,11). It can be represented as:

00 -> a or 0

01 -> b or 1

11 -> c or 2

10 -> d or 3

Same as a data type of n bit system can store up to 2^n values.

Swift 4 Data types

The most common Swift 4 data types are:

Bool Data type

The Bool data type can store only two values either true or false.

Its default value is False.

It is generally used in if-else statement.

Boolean Data type example:

```
let result:Bool = true
```

```
print(result)
```

Output:

```
true
```

Integer Data type

The integer data type can store both positive and negative numbers including zero with no fractional values.

Its default value is 0.

Its size depends on the platform type which may be 32 bits or 64 bits.

Range: -2,147,483,648 to 2,147,483,647 (32 bit platform)

-9223372036854775808 to 9223372036854775807 (64 bit platform)

Integer data type has many other variants such as UInt, Int8, Int16 etc. which can be used if you want to specify storage type specifically.

Integer Data type example:

```
var highScore:Int = 50
```

```
print(highScore)
```



```
highScore = 50  
print(highScore)
```

Output:

```
50  
40
```

Explanation: In the above program, we have declared a variable highScore of Int type and then assign a value of 50 so; it provides an output of 50.

Later, we changed the value to 40 using assignment operator highScore = 40 so, it provides the output 40.

There are some more variants of Int data type in Swift.

Int8 Data type:

Int8 data type can store both positive and negative small numbers. Its default value is 0, size is 8 bit and range varies from -128 to 127.

It can store 256 values means values from -128 to 127 including 0.

```
var highScore:Int8 = -128//ok  
highScore = 127 //ok  
highScore = 128 // error occurred  
highScore = -129 //error occurred
```

You can check the highest and lowest values Int8 data type can store by using .min and .max.

Example of .min and .max Int8 data type

```
print(Int8.min)  
print(Int8.max)
```

Output:

```
-128  
127
```

UInt Data type

UInt (Unsigned Integer) can only store unsigned numbers (positive and zero). Other remaining properties are same as Integer data type means:

Default Value: 0

Size: 32/64 bit depends on the platform type.

Range: 0 to 4294967295 (32 bit platform)

0 to 18446744073709551615 (64 bit platform)

Float Data type

The float data type can store decimal and fractional numbers. Its default value is 0.0, size 32 bit and range varies from 1.2×10^{-38} to 3.4×10^{38} (~6 digits).

Float data type example:

```
let highScore:Float = 3.1416
```

```
print(highScore)
```

Output:

3.1416

Double Data type

The Double data type can store decimal or fractional numbers as Float data type but larger decimal points than Float supports.

Its default value is 0.0, size: 64-bit, range varies from 2.3×10^{-308} to 1.7×10^{308} (~15 digits)

Double data type example:

```
let highScore:Double = 3.1416
```

```
print(highScore)
```

Output:

3.1416

Character Data type

Character data type can store a single character string literal. It is used to add emoji or languages other than English.

Character data type example:

```
let playerName:Character = "J"
```

```
let playerNameWithUnicode:Character = "\u{134}"
```

```
print(playerName)
print(playerNameWithUnicode)
```

Output:

```
J
Ĵ
```

String Data type

String data type can store collection of characters. It is Value type and its default value is "" (Empty String).

Swift facilitates you to use a few special escape sequences to use them in string. For example,

```
\0 (null character),
\\ (a plain backslash \),
\t (a tab character),
\v (a vertical tab),
\r (carriage return),
\" (double quote),
\' (single quote), and
\u{n} (unicode code point, n is in hexadecimal).
```

String Data type example:

```
let playerName = "Jack"
let playerNameWithQuotes = "\"Jack\""
let playerNameWithUnicode = "\u{134}ack"
print(playerName)
print(playerNameWithQuotes)
print(playerNameWithUnicode)
```

Output:

```
Jack
"Jack"
Ĵack
```

Range of data type variables in tabular form

Type	Typical Bit Width	Typical Range
Int8	1byte	-127 to 127
UInt8	1byte	0 to 255
Int32	4bytes	-2147483648 to 2147483647
UInt32	4bytes	0 to 4294967295
Int64	8bytes	-9223372036854775808 to 9223372036854775807
UInt64	8bytes	0 to 18446744073709551615
Float	4bytes	1.2E-38 to 3.4E+38 (~6 digits)
Double	8bytes	2.3E-308 to 1.7E+308 (~15 digits)

Swift variables

Variables are used to store data in memory so that we can use them in program. Variables are like container that can hold data which can be changed later. Every variable has a unique name called identifier.

How to declare Swift Variables?

In Swift 4, var keyword is used to declare variable.

Example:

```
var siteName:String  
print(siteName)
```

Here, we have declared a variable named `siteName` of type `String`. It can hold only string values. If you execute the above code, it will give a compile time error because we have only declared variable and not assigned any value.

Let's see how to assign values to a Swift variable.

How to assign values in Swift variable?

Assignment operator (`=`) is used to assign value in Swift variable.

Example:

```
var siteName:String
siteName = " ISMA/MACT "
print(siteName)
```

Or

```
var siteName:String = "ISMA/MACT"
print(siteName)
```

Output:

ISMA/MACT

Note: As, Swift is a type inferred language, it automatically knows "`javatpoint.com`" is a `String` and declare `siteName` as a `String`. You can even remove the type (`:String`) from declaration and it will show the same result.

Example:

```
var siteName = " ISMA/MACT "
print(siteName)
```

Output:

ISMA/MACT

How to change the value of Swift Variable?

We can change the value of a variable by using assignment operator but without `var` keyword.

Example:

```
var siteName = " ISMA/MACT "
```

```
// Assigning a new value to siteName
siteName = "ISMA/MACT.com"
print(siteName)
```

Output:

ISMA/MACT.com

Print Current value of variables

We can use the print function to print the current value of variable or constant. Wrap the value in parenthesis with a backslash before the opening parenthesis.

Example:

```
var varA = "Java"
var varB = 3
print("The course duration of \ (varA) is \ (varB) months.")
```

Output:

The course duration of Java is 3 months.

Swift Constants

As the name shows, constants are the special type of variables whose values can't be changed. They are like containers which are stored permanently with some items.

How to declare Swift Constants?

The let keyword is used to declare a variable.

Example:

```
let siteName:String
print(siteName)
```

Here, we have declared a constant named siteName of type String. It can hold only string values. If you execute the above code, it will give a compile time error because we have only declared the constant and not assigned any value.

Let's see how to assign values to a Swift constant.

How to assign values in Swift constant?

We can assign a value in Swift constant by using Assignment operator (=).

Example:

```
let siteName:String  
siteName = " ISMA/MACT "  
print(siteName)
```

Or

```
let siteName:String = " ISMA/MACT "  
print(siteName)
```

Output:

ISMA/MACT

As Swift is a type inferred language, you can remove the type (:String) from declaration and even it will show the correct result.

Example:

```
let siteName = " ISMA/MACT "  
print(siteName)
```

Output:

ISMA/MACT

Note: You can't change the value of the constant.

First Swift Program

To run your first Swift program, you must have installed Xcode and Swift on your computer.

Open the Xcode playground and write down the following code:

Program:

```
// This is the first Swift program also called Hello, World! Program  
import Swift  
print("Hello, World!")
```

Output:

"Hello World"

Program Explanation

1. // This is the first Swift program also called Hello, World! Program

The above line which is started with two slashes (//) is a comment. It is totally ignored by the compiler. Comments are used to make your code understandable by other person.

2. import Swift

The import Swift keyword is used to access all the symbols defined inside the Swift language. Without using this keyword, you can't use print statement to print anything on Xcode.

3. print("Hello, World!")

The above line is a print function in Swift. It is used to show the output on the screen.

The "Hello World" program on Terminal
Follow the steps given below:

Open the terminal

Type swift and press enter. You will get a welcome message as Welcome to Apple Swift version x.x.x.

Type print("Hello, World!")

Press Enter

Output:

Hello, World!

Note: Unlike other programming language C, C++, Java, Python etc., Swift doesn't need to add a semicolon (;) at the end of the statement. It is optional. You can add multiple lines in a single line by using semicolon (;) at the end of first statement.

Example:

```
print("Hello World",");print(" Welcome to ISMA/MACT")
```

Swift Operators

In Swift 4, an operator is special symbol or phrase which is used to check, change or combine values. It tells the compiler to perform specific mathematical or logical manipulations. For example, the addition operator (+) adds two numbers, the logical AND operator (&&) combines two Boolean values etc.

Swift 4 supports most of the standard C language operators. It enhances some capabilities to remove some common coding errors.

Classification of Operators on Terminology

There are three types of operators: unary, binary and ternary.

Unary operator: Unary operators are used on a single target (like -a). Unary prefix operators appear immediately before their target (such as !b). Unary postfix operators appear immediately after their target (such as c!).

Binary Operator: Binary operators are used when we have to operate on two targets. (like 2 + 3) and are infix because they appear in between their two targets.

Ternary Operator: Ternary operators operate on three targets. Like C language, Swift also has only one ternary operator called the ternary conditional operator (a ? b : c).

Note: The values associated with operator are operands. For example, in the expression 1 + 2, the + symbol is a binary operator and its two operands are the values 1 and 2.

The most common built-in operators in Swift 4

Assignment Operator

Arithmetic Operators

Remainder Operator

Unary Minus Operator

Unary Plus Operator

Compound Assignment Operators

Comparison Operators

Ternary Conditional Operator

Range Operators

Assignment Operator

Assignment operator is used to initialize or update the values of both its operands. For example, here (a = b), the assignment operator (=) initializes or updates the value of a with the value of b.

```
let b = 20
```

```
var a = 10
```

```
a = b
```

```
// a is now equal to 20
```

If the right side operand contains the multiple values, its elements can be decomposed into multiple constants or variables at once:

```
let (x, y) = (10, 20)
```

```
// x is equal to 10, and y is equal to 20
```

Swift 4 assignment operator does not itself return a value like C and Objective C assignment operators do.

So, the following statement is invalid in Swift 4:

```
if x = y {
```

```
    // This is not valid, because x = y does not return a value.
```

```
}
```

Arithmetic Operators

Swift 4 supports the four standard arithmetic operators of all number types:

Addition (+) Operator

Subtraction (-) Operator

Multiplication (*) Operator

Division (/) Operator

```
10 + 20    // equals 30
```

```
50 - 30    // equals 20
```

```
20 * 3     // equals 60
```

```
10.0 / 2.5 // equals 4.0
```

Swift 4 arithmetic operators don't allow values to overflow by default (like the arithmetic operators in C and Objective-C language do).

In Swift 4, we can opt in to value overflow behavior by using Swift's overflow operators (such as `&+ b`).

We can also use addition operator for String concatenation:

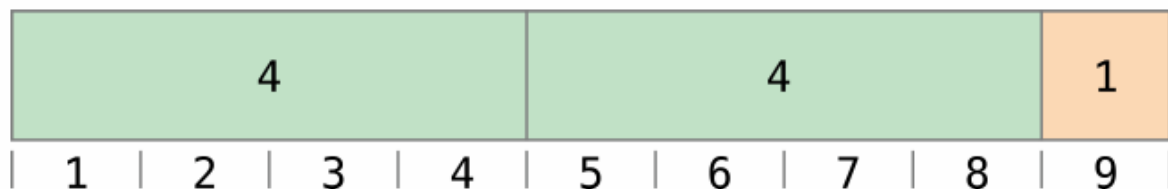
"hello, " + "world" // equals "hello, world"

Remainder Operator

Swift 4 remainder operator ($a\%b$) specifies how many multiples of the second operand (b) will fit inside first operand (a) and returns the value that is left over (known as the remainder).

Note: In other language, the remainder operator is known as modulo operator but in Swift 4, its behavior for negative numbers strictly specifies that it is a remainder rather than a modulo operation.

For example, to calculate $9\%4$, first find out how many 4s will fit inside 9.



There are two 4s inside 9 and 1 value is remaining. So, it will show 1 as output.

In Swift, it will be written as:

$9 \% 4$ // equals 1

For $a \% b$, the $\%$ operator calculates the following equation and returns remainder as output:

$$a = (b \times \text{multiplier}) + \text{remainder}$$

Here, multiplier is the largest number of multiples of b that will fit inside a .

$$9 = (4 \times 2) + 1$$

Same method for negative value of a

$$-9 \% 4 \text{ // equals } -1$$

Inserting -9 and 4 into the equation yields:

$$-9 = (4 \times -2) + -1$$

It gives a remainder value of -1.

The sign of b is ignored for negative values of b . This means that $a \% b$ and $a \% -b$ always give the same result.

Unary Minus Operator

The unary minus operator is a prefixed (-) sign attached with a value.

```
let three = 3
```

```
let minusThree = -three    // minusThree equals -3
```

```
let plusThree = -minusThree // plusThree equals 3, or "minus minus three"
```

The unary minus operator (-) is placed directly before the value, without any white space.

Unary Plus Operator

The unary plus operator (+) returns the exactly same value it operates on, without any change.

```
let three = 3
```

```
let plusThree = +3 (same as 3)
```

Compound Assignment Operators

Swift 4 provides compound assignment operators that combine assignment (=) operator with another operator. One example of compound assignment operator is the addition assignment operator (+=)

For example,

```
var a = 10
```

```
a += 20
```

```
// a is now equal to 30
```

Here, a += 20 is short form for a = a + 20. Addition and assignment operators are combined into one operator that performs both tasks at the same time.

Note: The compound assignment operators don't return a value. For example, you can't write let b = a += 20.

Comparison Operators

Swift 4 supports all standard comparison operators of C language.

Equal to (a == b)

Not equal to (a != b)

Greater than (a > b)

Less than (a < b)

Greater than or equal to (a >= b)

Less than or equal to (a <= b)

Note: Swift 4 also provides two identity operators (=== and !==), which is used to test whether both object references refer to the same object instance.

Every comparison operator returns a Bool value to specify whether the statement is true or not.

```
1 == 1 // true because 1 is equal to 1
```

```
2 != 1 // true because 2 is not equal to 1
```

```
2 > 1 // true because 2 is greater than 1
```

```
1 < 2 // true because 1 is less than 2
```

```
1 >= 1 // true because 1 is greater than or equal to 1
```

```
2 <= 1 // false because 2 is not less than or equal to 1
```

Comparison operators are used in conditional statement as if statement.

Example 1

```
let name = "world"
```

```
if name == "world" {
```

```
    print("hello, world")
```

```
} else {
```

```
    print("I am sorry. I don't recognize you")
```

```
}
```

```
// Prints "hello, world", because name is indeed equal to "world".
```

Output:

```
hello, world
```

Example 2

```
let name = "world"
```

```
if name == "JavaTpoint" {
```

```
    print("hello, world")
```

```
} else {
```

```
    print("I am sorry. I don't recognize you")
```

```
}
```

```
// Prints "I am sorry. I don't recognize you", because name is not equal to "world".
```

Output:

I am sorry. I don't recognize you

Ternary Conditional Operator

The ternary conditional operator is used to shorten the code of if statement. It is a special type of operator with three parts i.e. question ? answer1 : answer2

It checks whether the question is true or false. If the question is true, it will return the value of answer1; otherwise, it will return the value of answer2.

Ternary conditional operator is a short form of the following code:

```
if question {  
    answer1  
} else {  
    answer2  
}
```

Let's see the usage of ternary conditional operator in an example. In this example, we shall calculate the height for a table row. The row height should be 50 points taller than the content height if the row has a header and 20 points taller if the row doesn't have a header.

Example (Without Ternary Conditional Operator)

```
let contentHeight = 40  
let hasHeader = true  
let rowHeight: Int  
if hasHeader {  
    rowHeight = contentHeight + 50  
} else {  
    rowHeight = contentHeight + 20  
}  
// rowHeight is equal to 90
```

Output:

Example (Using Ternary Conditional Operator)

```
let contentHeight = 40
```

```
let hasHeader = true
let rowHeight = contentHeight + (hasHeader ? 50 : 20)
// rowHeight is equal to 90
```

Range Operators

Swift 4 supports several types of range operators which are used as shorthand for expressing a range of values.

Closed Range Operators

Half-Open Range Operator

One Sided Ranges

Closed Range Operator

The closed range operator (`a...b`) specifies the range between `a` and `b` (here `a` and `b` inclusive and value of `a` should not be greater than value of `b`). The closed range operator is useful when you have to iterate over a range in which you want all of the values to be used, such as with a `for-in` loop.

Example:

```
for index in 1...10 {
    print("\(index) times 5 is \(index * 5)")
}
```

Output:

```
1 times 5 is 5
2 times 5 is 10
3 times 5 is 15
4 times 5 is 20
5 times 5 is 25
6 times 5 is 30
7 times 5 is 35
8 times 5 is 40
9 times 5 is 45
10 times 5 is 50
```

Half-Open Range Operator

The half-open range operator (`a..b`) specifies a range between `a` to `b`, but it doesn't include `b`. It is called half-open range operator because it contains its

first value, but not its final value. Similarly as closed range operator, the value of a must not be greater than b in half-open range operator also. If the value of a is equal to b, then the resulting range will be empty.

Half-open range operator is generally used with arrays where it counts the length of the list.

Example:

```
let names = ["Albert", "Aryan", "Ajeet", "Jill"]
let count = names.count
for i in 0..
```

Output:

Person 1 name is Albert

Person 2 name is Aryan

Person 3 name is Ajeet

Person 4 name is Jill

One Sided Ranges

One sided ranges operator is an alternative form of closed range operator or half-open range operator in one direction only.

Example1:

```
let names = ["Albert", "Aryan", "Ajeet", "Jill"]
let count = names.count
for name in names[2...] {
    print(name)
}
```

Output:

Ajeet

Jill

Example2:

```
let names = ["Albert", "Aryan", "Ajeet", "Jill"]
let count = names.count
for name in names[...2] {
```



```
    print(name)
}
```

Output:

Albert
Aryan
Ajeet

Example3:

```
let names = ["Albert", "Aryan", "Ajeet", "Jill"]
let count = names.count
for name in names[..<2] {
    print(name)
}
```

Output:

Albert
Aryan

Swift Switch Statement

The Switch statement is used as a substitute for long if-else-if statement while matching complex patterns. It provides multiple cases to perform different actions according to different conditions.

Syntax:

```
switch variable/expression {
case value1:
    // statements
case value2:
    // statements
default:
    // statements
}
```

Example

```
let dayOfWeek = 6
switch dayOfWeek {
case 1:
```

```

        print("It is Sunday today")
    case 2:
        print("It is Monday today")
    case 3:
        print("It is Tuesday today")
    case 4:
        print("It is Wednesday today")
    case 5:
        print("It is Thursday today")
    case 6:
        print("It is Friday today")
    case 7:
        print("It is Saturday today")
    default:
        print("Invalid day")
}

```

Output:

It is Friday today

Swift If Statement

The if statement is a control flow statement which is used when we want to perform different actions according to some specified conditions which are either true or false.

Syntax:

```

if expression {
    // statements
}

```

Example: (if condition is true)

```

let number = 5
if number > 0 {
    print("This is a positive number.")
}
print("This will be executed anyways.")

```

Output:

This is a positive number.

This will be executed anyways.

Example: (if condition is false)

If we initialise the value with negative number like -5 and the test condition is same then the test expression will evaluate false. So, the statement inside the code block of if is skipped without execution.

```
let number = -5
if number > 0 {
    print("This is a positive number.")
}
print("This will be executed anyways.")
```

Output:

This will be executed anyways.

Swift if-else Statement

The Swift if-else statement contains two statements: if statement and else statement.

If the test evaluation is true, the if statement is executed and if the the test evaluation is false, then the else statement is executed.

Syntax:

```
if expression {
    // statements
} else {
    // statements
}
```

Example:

```
let number = 5
if number > 0 {
    print("This is a positive number.")
}
```

```
} else {  
    print("This is not a positive number.")  
}  
print("This will be executed anyways.")
```

Output:

This is a positive number.

This will be executed anyways.

the constant number value to negative (-5) and the test condition is same.

```
let number = -5  
if number > 0 {  
    print("This is a positive number.")  
} else {  
    print("This is not a positive number.")  
}  
print("This will be executed anyways.")
```

Output:

This is not a positive number.

This will be executed anyways.

Swift if-else-if Statement

The if-else-if statement is used when you want to execute one block of code among many.

Syntax:

```
if expression1 {  
    // statements  
}  
else if expression2 {  
    // statements  
}  
else if expression3 {  
    // statements  
}
```

```
.  
.
else {
    // statements
}
```

Example:

```
let number = 0;
if number > 0 {
    print("This is a positive number.")
}
else if (number < 0) {
    print("This is a negative number.")
}
else {
    print("This number is 0.")
}
```

Output:

This number is 0.

Swift Nested if-else Statement

Swift language facilitates you to use an if-else statement inside an if-else statement. Nested if-else statement can also be used in switch statement in Swift.

Example:

```
let n1 = 10, n2 = 22, n3 = 25
if n1 >= n2 {
    if n1 >= n3 {
        print("The largest number is ", n1)
    }
    else {
        print("The largest number is ", n3)
    }
}
else {
```

```

if n2 >= n3 {
    print("The largest number is ", n2)
}
else {
    print("The largest number is ", n3)
}
}

```

Output:

The largest number is 25

Swift Guard Statement

Swift Guard statement is used as a substitute of Swift if statement. Guard statement provides benefits over if statement to control the program flow and write simple and clean code.

Syntax:

```

guard expression else {
    //statements
    //must contain a control statement: return, break, continue or throw.
}

```

Example:

```

guard true else {
    print("Condition is not satisfied.")
}
print("Condition is satisfied.")

```

Output:

Condition is satisfied.

Example 2:

In Swift, we can also use a guard statement in function.

```

func Function1() {
    guard false else {

```

```

        print("Condition is not satisfied.")
        return
    }
    print("Condition is satisfied.")
}
Function1()
print("Hello after function call")

```

Output:

Condition is not satisfied.
Hello after function call

Break Statement

Swift 4 break statement is used in two cases:

The break statement is used within a loop when you have to immediately terminate statement. The program control resumes at the next statement following the loop.

It is also used to terminate a case in switch statement.

In the case of nested loops, the break statement terminates the inner most loop and start executing the next line of the code after the block.

Syntax:

The syntax of Swift 4 break statement is:

```
break
```

Example:

```
var index = 10
```

```
repeat {
    index = index + 1
    if( index == 25 ){
        break
    }
    print( "Value of index is \(index)")
} while index < 30
```

Output:

Value of index is 11
Value of index is 12
Value of index is 13
Value of index is 14
Value of index is 15
Value of index is 16
Value of index is 17
Value of index is 18
Value of index is 19
Value of index is 20
Value of index is 21
Value of index is 22
Value of index is 23
Value of index is 24

Continue Statement

Swift 4 continue statement is used to stop currently executing statement and start again at the beginning of the next iteration through the loop. The continue statement is used with for loop, while loop and do... while loop.

Syntax:

The syntax of the continue statement for the Swift 4 loops is as follow:

continue

Example:

```
var index = 10
```

```
repeat {  
    index = index + 1  
    if( index == 25 ){  
        continue  
    }  
    print( "Value of index is \(index)")  
} while index < 30
```


Output:

Value of index is 11
Value of index is 12
Value of index is 13
Value of index is 14
Value of index is 15
Value of index is 16
Value of index is 17
Value of index is 18
Value of index is 19
Value of index is 20
Value of index is 21
Value of index is 22
Value of index is 23
Value of index is 24
Value of index is 25
Value of index is 26
Value of index is 27
Value of index is 28
Value of index is 29
Value of index is 30

Fallthrough Statement

Swift 4 fallthrough statement is used to simulate the behavior of Swift 4 switch to C/ C++ style switch. In Swift 4, the switch statement completes its execution as soon as the first matching case is completed unlike the C and C++ programming languages where falling through the bottom of subsequent cases happen.

Syntax of Switch Statement in C/C++

```
switch(expression){  
    case constant-expression :  
        statement(s);  
        break; /* optional */  
    case constant-expression :  
        statement(s);
```

```

    break; /* optional */

    /* you can have any number of case statements */
    default : /* Optional */
        statement(s);
}

```

In the above code, we need a break statement to come out of a case statement, otherwise the execution control will fall through the subsequent case statements available below the matching case statement.

Syntax of Switch Statement in Swift 4

```

switch expression {
    case expression1 :
        statement(s)
    fallthrough /* optional */
    case expression2, expression3 :
        statement(s)
    fallthrough /* optional */

    default : /* Optional */
        statement(s);
}

```

In the above code, if we don't use fallthrough statement, then the program will come out of the switch statement after executing the matching case statement.

Let's see an example to make it clear.

Example: (Swift 4 example with fallthrough statement)

Let's see how to use a switch statement in Swift 4 without fallthrough statement:

Example 1:

```
var index = 4
```

```
switch index {
```

```

case 1:
    print( "Hello Everyone")
case 2,3 :
    print( "This is JavaTpoint")
case 4 :
    print( "JavaTpoint is an educational portal")
default :
    print( "It is free to everyone")
}

```

Output:

JavaTpoint is an educational portal

Example 2:

Let's see how to use a switch statement in Swift 4 programming with fallthrough statement.

```

var index = 10
switch index {
case 100:
    print( "Hello Everyone")
    fallthrough
case 10,15 :
    print( "This is JavaTpoint")
    fallthrough
case 5 :
    print( "JavaTpoint is an educational portal")
default :
    print( "It is free to everyone")
}

```

Output:

This is JavaTpoint

JavaTpoint is an educational portal

Swift 4 Loops

Loop is used when you want to execute a block of codes several times. It executes the statement sequentially. It means it executes the first statement within the function first, followed by second and third and so on.

Loops used in Swift 4 programming language

A list of most used loops in Swift 4 programming language:

for-in loop: The for-in loop is used to execute a set of statements for each item in a range, selection, collection etc.

while loop: The while loop is used to repeat a statement or a set of statements when the given condition is true. It checks the conditions before executing the loop body.

repeat... while loop: It is same as the while loop but it checks the condition at the end of the loop body.

Swift For-in Loop

The Swift For-in loop is used to run a set of tasks for a certain number of times. This loop does repetitive process a number of times.

Example 1:

```
let names = ["Ajeet", "Aryan", "Ayan", "Alex"]
for name in names {
    print("Good Morning \(name)")
}
```

Output:

```
Good Morning Ajeet
Good Morning Aryan
Good Morning Ayan
Good Morning Alex
```

Swift While and Repeat While Loop

While and Repeat while loops are used as an alternative of for-in loops when the number of iteration is not known. The while loop executes a set of

statements until a false condition occur. This loop is generally used when you don't know about the number of iterations.

There are two types of loops in Swift:

While loop

Repeat While loop

Swift While Loop

The Swift while loop evaluates its condition at the start of each pass.

Syntax:

```
while (TestExpression) {  
    // statements  
}
```

Example:

```
var currentLevel:Int = 0, finalLevel:Int = 6  
let gameCompleted = true  
while (currentLevel <= finalLevel) {  
    //play game  
    if gameCompleted {  
        print("You have successfully completed level \(currentLevel)")  
        currentLevel += 1  
    }  
}  
//outside of while loop  
print("Terminated! You are out of the game ")
```

Output:

```
You have successfully completed level 0  
You have successfully completed level 1  
You have successfully completed level 2  
You have successfully completed level 3  
You have successfully completed level 4  
You have successfully completed level 5  
You have successfully completed level 6  
Terminated! You are out of the game
```

Repeat While loop

The Repeat While loop is same as while loop but a difference that the body of repeat...while loop is executed once before the test expression is checked.

Syntax:

```
repeat {  
    // statements  
    ...  
} while (testExpression)
```

Example:

```
var currentLevel:Int = 0, finalLevel:Int = 5  
let gameCompleted = true  
repeat {  
    //play game  
    if gameCompleted {  
        print("You have successfully completed level \$(currentLevel)")  
        currentLevel += 1  
    }  
} while (currentLevel <= finalLevel)  
print("Terminated! outside of repeat while loop")
```

Output:

```
You have successfully completed level 0  
You have successfully completed level 1  
You have successfully completed level 2  
You have successfully completed level 3  
You have successfully completed level 4  
You have successfully completed level 5  
Terminated! outside of repeat while loop
```

Swift Arrays

Arrays are used to store multiple values of same data type. An Array is just like a container which holds multiple data/ values of a data type in an ordered list. When you fetch the data, you will get the elements in the same order as you defined in the array.

An array can store values of any data type e.g. Int, String, class etc.

How to declare array in Swift 4?

Specify the data type inside the square brackets [] to create an empty array. You have to specify the type inside the square bracket [], otherwise Swift will treat it as a normal data type and you can store only a single value in it.

Declare an empty array

Syntax:

```
let emptyIntArr:[Int] = []  
print(emptyIntArr)  
Or
```

```
let emptyIntArr:Array<Int> = Array()  
print(emptyIntArr)
```

Output:

```
[]
```

Here, we have declared a constant emptyIntArr that can store array of integer and initialized with 0 values.

Create an array directly

Swift is a type inference language so, we can also create an array directly without specifying the data type but we have to initialise with some values so that compiler can find out its type.

Example:

```
let someIntArr = [1, 2, 3, 4, 5]
print(someIntArr)
```

Output:

[1, 2, 3, 4, 5]

Declare an array with specified number of single repeated value

We can repeat a value to specific number of times to make an array in Swift. It is done by using the array initializer with repeating and count.

Example:

```
let arrWithRepeatingValues = Array(repeating: "ISMA/MACT", count: 5)
print(arrWithRepeatingValues)
```

Output:

["ISMA/MACT ", " ISMA/MACT ", " ISMA/MACT ", " ISMA/MACT ", " ISMA/MACT "]

How to store values in Array?

Let's take a constant IntArray to store some strings:

```
let intArray = [10, 11, 12, 13, 14, 15]
```

0	1	2	3	4	5
10	11	12	13	14	15

A list of built in Array functions and their properties:

Index	Array function	Explanation
1.	isEmpty	The isEmpty property is used to determine wheather an array is empty or not. It returns true if the array does not contain any value otherwise, returns false.
2.	first	This property is used to access first element of an array.
3.	append	The append function is used to insert/append element at the end of the array.
4.	insert	This function is used to insert/append element at specific index of the array.
5.	remove	This function removes and returns the value specified at the specific position from the array.
6.	reversed	This function returns the elements of array in reverse order.
7.	count	This property returns the total number of elements in an array.

Swift Dictionary

A Swift dictionary is a simple container that can contain multiple data as key-value pair in an unordered way.

Swift dictionary is used instead of array when you want to look up value with some identifier in the collection. Suppose, we have to search the capital city of country. In this case, we will create a dictionary with key country and value capital city. Now, you get the capital city from the collection by searching with the key country. Here, we have paired a country to its capital city.

Dictionary declaration in Swift

Declaring an empty dictionary

To create an empty dictionary, we specify the key:value Data type inside square brackets [].

Example:

```
let emptyDictionary:[Int:String] = [:]
```

```
print(emptyDictionary)
```

Or

```
let emptyDictionary:Dictionary<Int, String> = [:]  
print(emptyDictionary)
```

Output:

```
[:]
```

Declaring a dictionary with some values

```
let valDictionary = ["a":10, "b":20, "c":30, "d":40, "e":50, "f":60, "g":70, "h":80,  
"i":90]  
print(valDictionary)
```

Output:

```
["c": 30, "d": 40, "g": 70, "b": 20, "a": 10, "f": 60, "h": 80, "i": 90, "e": 50]
```

In the above program, we have not declared the type explicitly but we initialize with some default elements. The element is in key:value pair where key is of type String and value is of Int type.

Creating Dictionary from two arrays

We can also create dictionary from arrays.

Example:

```
let customKeys = ["India", "Pakistan", "United Kingdom"]  
let customValues = ["New Delhi", "Islamabad", "London"]  
let newDictionary = Dictionary(uniqueKeysWithValues:  
zip(customKeys,customValues))  
print(newDictionary)
```

Output:

```
["India": "New Delhi", "United Kingdom": "London", "Pakistan": "Islamabad"]
```

Access Dictionary elements in Swift

We can access elements of a Swift dictionary by including key of the value we want to access within square brackets immediately after the name of the dictionary.

Example:

```
let valDictionary = ["a":10, "b":20, "c":30, "d":40, "e":50, "f":60, "g":70, "h":80, "i":90]
print(valDictionary["c"])
print(valDictionary["f"])
```

Output:

Optional(30)

Optional(60)

Access Dictionary elements in Swift using for-in loop

Example:

```
let valDictionary = ["a":10, "b":20, "c":30, "d":40, "e":50, "f":60, "g":70, "h":80, "i":90]
for (key,value) in valDictionary {
    print("key:\(key) value:\(value)")
}
```

Output:

key:h value:80

key:d value:40

key:a value:10

key:g value:70

key:e value:50

key:b value:20

key:c value:30

key:f value:60

key:i value:90

Modify dictionary elements in Swift

We can add a new element in a Swift dictionary by using a new key as index and assign to a new value.

Example:

```
var valDictionary = ["India":"New Delhi", "China":"Beijing", "USA":"Washington D.C."]
valDictionary["Germany"] = "Berlin"
print(valDictionary)
```

Output:

```
[ "Germany": "Berlin", "China": "Beijing", "USA": "Washington D.C.", "India": "New Delhi"]
```

Changing elements in Dictionary

Example:

```
var valDictionary = [ "India": "New Delhi", "China": "Beijing", "USA": "Washington D.C." ]
```

```
valDictionary["India"] = "NEW DELHI"
```

```
print(valDictionary)
```

Output:

```
[ "USA": "Washington D.C.", "China": "Beijing", "India": "NEW DELHI" ]
```

Swift Functions

Functions are the set of codes which are used to perform a specific task. In Swift 4, a function is used to pass local as well as global parameter values inside the function calls.

Functions have a specific name which is used to "call" the function to perform its task when needed.

Swift 4 functions contain parameter type and its return types.

Types of Functions

Types of functions depend on whether a function is predefined or created by programmer. In Swift 4, there are two types of functions:

Library functions (Built-in Functions) - Library functions are the type of functions which are defined already in Swift Framework.

User-defined functions - User defined functions are created by the programmer themselves.

Library Functions

Library functions are the user-defined functions which are already defined in Swift framework. These functions are used to solve common problems and simple operations in Swift like printing, finding minimum and maximum, etc. so that you don't have to solve them yourselves.

Library functions can be used directly by invoking (calling) it. For example:
print() function

You can see all the functions inside the Swift framework by importing Swift. Open command prompt and click on it. You will see a new page. Search all the statements starting with func keyword.

Example:

```
print("ISMA/MACT")
```

User-defined Functions

Swift 4 facilitates you to define your own functions. User defined functions are mainly used to solve problems or perform tasks not available in Swift Framework. These functions can also be reused to perform similar tasks in the future.

Function Syntax

```
func function_name(args...) -> ReturnType {  
    //statements  
    return value  
}
```

How to call function in Swift 4?

Example:

```
func wish(user:String) {  
    print("Good Morning \(user). Have a Good day.")  
}  
wish(user: "Ajeet")
```

Output:

Good Morning Ajeet. Have a Good day.

Swift Nested Function

A function inside the body of another function is called nested function.

Syntax:

```
func function1() {
```

```

//statements of outer function
func function2() {
    //statements of inner function
}
}

```

Nested Function without Return Values

```

func wish1(_ message: String) { // parameter is given with message
    func wish2() { //function is calling
        print("Good Morning! \"(message)\")
    }
    wish2 ()
}
wish1("Ajeet") //giving the name of the message

```

Output:

Good Morning! Ajeet

Nested function with parameters and Return value

The nested functions can also contain parameters and return values.

Example:

```

func operate(with symbol:String) -> (Int, Int) -> Int {
    func add(num1:Int, num2:Int) -> Int {
        return num1 + num2 //10+20=30
    }
    func subtract(num1:Int, num2:Int) -> Int {
        return num1 - num2 //10-20=-10
    }
    let operation = (symbol == "+") ? add : subtract
    return operation
}
let operation = operate(with: "+")
let operation =operate(with:"-")
let result = operation(10, 20)
print(result)

```

Output:

30

10

Swift Function Parameter and Return Value

The Swift 4 user-defined function can have multiple parameters and different return values.

a) Functions with no parameter and no return value

The Swift 4 functions with no parameter and no return value do not take any input and return value.

Syntax:

```
func funcname() {  
    //statements  
}
```

OR

```
func funcname() -> () {  
    //statements  
}
```

OR

```
func funcname() -> Void {  
    //statements  
}
```

Example:

```
func greeting() {  
    print("Good Morning Everyone")  
}  
greeting()
```

Output:

Good Morning Everyone

b) Functions with no parameters but having return value

Example:

```
func greeting() -> String {  
    return "Good Morning Everyone"  
}  
let msg = greeting()
```

```
print(msg)
```

Output:

Good Morning Everyone

c) Function with Parameter but having no return value

In Swift 4, some functions have parameters but don't have return value. Parameters are used to take input in the function.

Syntax:

```
func funcname(parameterName:Type) {  
    //statements  
}
```

Example:

```
func greeting(msg:String) {  
    print(msg)  
}  
greeting(msg: "Good Morning Everyone")
```

Output:

Good Morning Everyone

d) Functions with Parameter and Return Value

When a function takes parameters and also returns value.

Syntax:

```
func funcname(parameterName:Type) -> ReturnType {  
    //statements  
}
```

Example:

```
func greeting(name:String) -> String {  
    return "Welcome to ISMA/MACT " + name  
}  
let msg = greeting(name: "Ajeet")  
print(msg)
```

Output:

Welcome to ISMA/MACT Ajeet

e) Function with multiple parameters and multiple return values

These functions are used to take multiple parameters separated by comma and return multiple return values. In Swift, tuples are used to return multiple values.

Syntax:

```
func funcname(parameterName:Type, parameterName2:Type ,...) ->
(ReturnType, ReturnType...) {
    //statements
}
```

Example:

```
func greeting(name:String, coin:Int) -> (String, Int) {
```

```
    let msg = "Hello Mr./Ms." + name
```

```
    var userCoin = coin
```

```
    if coin < 0 {
```

```
        userCoin = 0
```

```
    }
```

```
    return (msg, userCoin)
```

```
}
```

```
let msg = greeting(name: "Ajeet", coin: 20)
```

```
print(msg.0)
```

```
print("You have \(msg.1) coins left in your wallet.")
```

Output:

Hello Mr./Ms.Ajeet

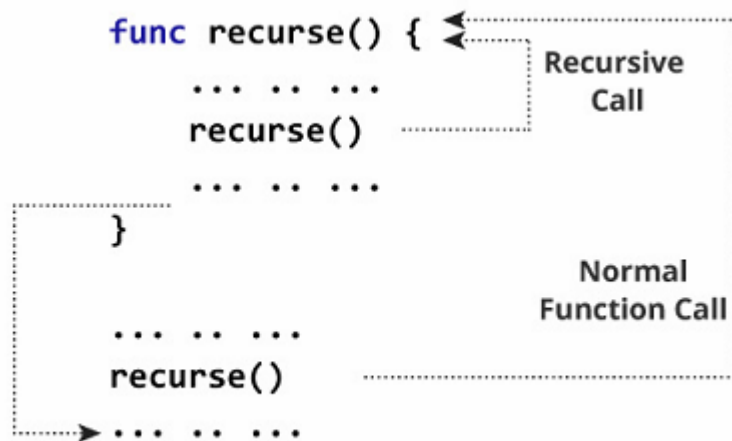
You have 20 coins left in your wallet.

Swift Recursion

A function that calls itself repeatedly is known as recursive function and this technique is called recursion. When you create a recursive function, you must create a condition so that the function does not call itself indefinitely.

Example:

```
func recurse() {
    //statements
    recurse()
}
recurse()
```



Swift Enum

Swift Enumeration is also known as Swift Enum. It is a data type which consists of set of related values. It is declared in a class and its values are accessed through the instance members of that class.

Syntax:

```
enum enum_name
{
    // values are described here
}
```

Example:

Let's take an example to understand it well. Here, we shall an enum named "Connection Status" and see the possible condition using switch statement.

```
enum ConnectionStatus
{
    case Unknown
    case Disconnected
    case Connecting
    case Connected
}
```

```

}
var connectionStatus = ConnectionStatus.Connecting
connectionStatus = .Connected

switch connectionStatus {

    case .Disconnected:
        print("Disconnected")

    case .Connecting:
        print("Connecting")

    case .Connected:
        print("Connected")

    default:
        print("Unknown Status")
}

```

Output:

Connected

Swift Structures

Swift structures are the flexible basic building blocks of the programs. The "struct" keyword is used to define structures. By using structures, you can define constructs methods and properties.

Syntax:

```

Struct StructureName
{
    // Definition of structure
}

```

Definition of a structure

Suppose, you have to access your bank account having some amount in rupees and paisa here, Here Amount is structure's name which is used to initialize a structure with rupees and paisa as datatype 'Int'

```
struct Amount
{
    var rupees: Int
    var paisa: Int
}
```

Access the structure and its properties

Let's see the implementation of the structures in Swift. Here, we shall assign some values in structure and then access them.

Example:

```
struct Amount
{
    var rupees: Int
    var paisa: Int
}
var x = Amount (rupees: 10000, paisa: 50)
print(x)
print("Amount in rupees is \(x.rupees)")
print("Amount in paisa is \(x.paisa)")
```

Output:

Amount(rupees: 10000, paisa: 50)

Amount in rupees is 10000

Amount in paisa is 50

Swift Classes

Classes in Swift are similar to Structures in Swift. These are building blocks of flexible constructs. You can define class properties and methods like constants, variables and functions are defined. In Swift 4, you don't need to create interfaces or implementation files while declaring classes. Swift 4 facilitates you to create classes as a single file and the external interfaces will be created by default when the class is initialized.

Syntax

Class classname

```
{  
// Definition goes here..  
}
```

Example

In the following program there are two classes Salary and EmpSalary.

```
class Salary{  
    var salary: Int  
    init(salary: Int) {  
        self.salary = salary  
    }  
}  
class EmpSalary {  
    var salary = 10000  
}  
let Income = EmpSalary()  
print("Employee salary is \(Income.salary)")
```

Output:

After running the above program, you will see the following output:

Employee salary is 10000

Inheritance in Swift

In Swift programming language, a class can inherit properties, methods and other characteristics from another class. Inheriting these properties and attributes from one class to another class is known as inheritance.

Sub class: The class which inherits the properties from other class is called child class or sub class.

Super class: The main class from where the subclass inherits the properties is known as parent class or super class.

Swift4 classes consist of superclass which calls and accesses methods, properties, functions and overriding methods. Property observers are used to add a property and modify the stored and computed property methods.

Syntax:

```
class class_name: Superclass_name
{
// Definition of the class
}
```

Base Class

A Base Class is a class which does not inherit methods, properties or functions from another class.

Example

```
class StudentDetails {
    var stname: String!
    var mark1: Int!
    var mark2: Int!
    var mark3: Int!
    init(stname: String, mark1: Int, mark2: Int, mark3: Int) {
        self.stname = stname
        self.mark1 = mark1
        self.mark2 = mark2
        self.mark3 = mark3
    }
}

let stname = "Aryan"
let mark1 = 70
let mark2 = 80
let mark3 = 85
print(stname)
print(mark1)
print(mark2)
print(mark3)
```

Output:

Aryan

70

80

Subclass

The subclass inherits properties, methods and functions of its base class. To define a subclass ':' is used before the base class name.

Example

```
class Person {  
  var firstName: String  
  var lastName: String  
  let birthPlace: String  
  
  init(birthPlace: String) {  
    self.birthPlace = birthPlace  
  }  
}  
  
class Student: Person  
{  
  var school: String  
}  
  
let student = Student(birthPlace: "India")  
student.lastName = "Head"  
student.firstName = "Travis"  
print(student.birthPlace)  
print(student.firstName)  
print(student.lastName)
```

Output:

India

Travis

Head

Overriding

Overriding is used to access the super class instance, type methods, instance, type properties and subscripts. It uses a keyword 'override' to override the methods declared in the superclass.

Use of super keyword

'super' keyword is used as a prefix to access the methods, properties and subscripts declared in the super class.

Overriding: Access to methods, properties and subscripts

Methods: `super.somemethod()`

Properties: `super.someProperty()`

Subscripts: `super[someIndex]`